

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) mitigates LLM hallucinations by grounding responses in retrieved external knowledge. While traditional vector RAG excels at factoid lookups, it struggles with global questions requiring cross-document reasoning. GraphRAG addresses this by constructing knowledge graphs from text corpora, enabling structured multi-hop retrieval [1, 2].

A critical limitation of current GraphRAG systems is their reliance on *full rebuilds* for corpus updates. Microsoft GraphRAG [1] requires re-running entity extraction, community detection, and summary generation over the entire corpus when any new document arrives. For a 1M-token corpus, this costs over 280 minutes of LLM time. LightRAG [2] introduced a set-union incremental update, but its naive merging creates duplicate entities and fragmented graph structure without deduplication, conflict resolution, or community repartitioning.

We propose IGV (Incremental Graph Versioner), which addresses these gaps through three innovations:

- **Semantic entity deduplication** using sentence-transformer embeddings with cosine similarity, reducing entity count by 13–17%.
- **Multi-hop graph relinking** that connects newly inserted entities to the existing graph via BFS-based candidate generation.
- **Incremental community repartitioning** that locally updates Louvain community assignments for affected nodes only.

Experiments on five datasets demonstrate that IGV achieves comparable retrieval quality (R@5 within 2%) to full rebuild while requiring only 0.3–0.8s per incremental batch. Statistical significance tests confirm F1 improvements over naive incremental ($p < 0.05$), and long-term streaming over 12 batches shows sub-9% degradation with near-zero forgetting rate.

2 RELATED WORK

GraphRAG Systems. Edge et al. [1] proposed GraphRAG, which builds entity knowledge graphs via LLM extraction, applies Leiden community detection, and generates community summaries for global query answering. LightRAG [2] simplified this with a dual-level retrieval paradigm and basic incremental updates via set union. PathRAG [3] introduced flow-based path pruning to reduce retrieval redundancy. HippoRAG [4] used Personalized PageRank for neurobiologically inspired retrieval.

Incremental Knowledge Graph Updates. GORAG [5] proposed online graph updates for social media classification but lacks stateful graph index evolution. KAG [6] introduced logical-form-guided reasoning with schema constraints but does not address incremental community repartitioning. Practical GraphRAG [7] reduced construction cost via dependency parsing but still requires full rebuilds for updates.

Community Detection. Louvain [8] and Leiden [9] algorithms are widely used for modularity-based community detection. Dynamic community detection has been studied in network science but has not been applied to GraphRAG index management.

3 METHODOLOGY

3.1 Problem Formulation

Given a knowledge graph $G = (V, E)$ constructed from an initial document corpus $\mathcal{D}_0$, and a stream of new document batches $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_T$, the goal is to update G to incorporate new entities and relationships from each batch $\mathcal{D}_t$ while minimizing LLM calls, avoiding duplicate entities, maintaining community structure quality, and preserving retrieval performance.

3.2 IGV Architecture

IGV processes each incremental batch through four stages:

Stage 1: Entity Extraction. For each new document, an LLM (Qwen2.5-7B-Instruct) extracts named entities and their relationships as triplets (s, r, o) .

Stage 2: Semantic Deduplication. New entities are compared against existing graph nodes using sentence-transformer embeddings ($d = 384$). Given a new entity e_{new} with embedding $\mathbf{e}_{\text{new}}$ and existing entities with embeddings $\{\mathbf{e}_1, \dots, \mathbf{e}_N\}$, we compute cosine similarity. If $\text{sim}(e_{\text{new}}, e_i) \geq \tau$ (threshold $\tau = 0.85$), entities are merged: the new entity’s relationships are redirected to the existing entity, and descriptions are concatenated.

Stage 3: Graph Relinking. After merging, we identify candidate edges between new and existing nodes using BFS expansion to depth $h = 2$. For each new node v_{new} , we explore its h -hop neighborhood and add edges to existing nodes with confidence ≥ 0.85 , where confidence is defined as $1/(1 + \text{dist}(v_{\text{new}}, v_{\text{old}}))$.

Stage 4: Incremental Community Repartitioning. Rather than recomputing communities for the entire graph, we extract the subgraph induced by affected nodes (new nodes + their 2-hop neighbors) and re-apply Louvain only on this subgraph. The new sub-communities are merged back into the global partition.

Table 1: Main results at Stage 3 (mean $\pm$ std over 3 seeds). Best in bold.

Dataset	Method	R@2	R@5	F1	EM	Comp	Div
HotpotQA	B1 Rebuild	0.202$\pm$.016	0.396$\pm$.013	0.108 $\pm$.004	0.040	2.5	2.0
	B2 Naive	0.202 $\pm$.014	0.386 $\pm$.004	0.103 $\pm$.003	0.020	2.4	1.9
	B3 +Dedup	0.186 $\pm$.019	0.379 $\pm$.000	0.107 $\pm$.003	0.020	2.5	2.0
	B4 +Relink	0.186 $\pm$.019	0.379 $\pm$.000	0.107 $\pm$.001	0.020	2.5	2.0
	IGV (Ours)	0.186 $\pm$.019	0.379 $\pm$.000	0.108$\pm$.002	0.020	2.5	2.0
2Wiki	B1 Rebuild	0.188 $\pm$.012	0.301 $\pm$.015	0.107 $\pm$.004	0.020	1.6	1.3
	B2 Naive	0.192$\pm$.006	0.321$\pm$.004	0.106 $\pm$.005	0.020	1.6	1.4
	B3 +Dedup	0.187 $\pm$.010	0.306 $\pm$.007	0.112$\pm$.008	0.020	1.6	1.4
	B4 +Relink	0.187 $\pm$.010	0.306 $\pm$.007	0.112$\pm$.008	0.020	1.6	1.4
	IGV (Ours)	0.187 $\pm$.010	0.306 $\pm$.007	0.111 $\pm$.009	0.020	1.6	1.4
MuSiQue	B1 Rebuild	0.080$\pm$.008	0.120$\pm$.008	0.036$\pm$.003	0.000	1.4	1.3
	B2 Naive	0.080$\pm$.000	0.117 $\pm$.005	0.031 $\pm$.005	0.000	1.4	1.3
	B3 +Dedup	0.073 $\pm$.005	0.113 $\pm$.005	0.034 $\pm$.004	0.000	1.4	1.3
	B4 +Relink	0.073 $\pm$.005	0.113 $\pm$.005	0.034 $\pm$.004	0.000	1.4	1.3
	IGV (Ours)	0.073 $\pm$.005	0.113 $\pm$.005	0.033 $\pm$.005	0.000	1.4	1.3
NarrativeQA	B1 Rebuild	0.162$\pm$.008	0.302$\pm$.010	0.089 $\pm$.004	0.040	2.3	1.8
	B2 Naive	0.158 $\pm$.004	0.295 $\pm$.012	0.087 $\pm$.004	0.020	2.2	1.7
	B3 +Dedup	0.139 $\pm$.013	0.284 $\pm$.005	0.086 $\pm$.002	0.020	2.2	1.8
	B4 +Relink	0.139 $\pm$.013	0.284 $\pm$.005	0.092$\pm$.002	0.020	2.2	1.7
	IGV (Ours)	0.139 $\pm$.013	0.284 $\pm$.005	0.091 $\pm$.001	0.020	2.2	1.8
StreamingQA	B1 Rebuild	0.044 $\pm$.000	0.065 $\pm$.000	0.065$\pm$.000	0.000	2.1	1.8
	B2 Naive	0.044 $\pm$.000	0.065 $\pm$.000	0.065$\pm$.000	0.000	2.2	1.8
	B3 +Dedup	0.044 $\pm$.000	0.065 $\pm$.000	0.064 $\pm$.001	0.000	2.1	1.7
	B4 +Relink	0.044 $\pm$.000	0.065 $\pm$.000	0.064 $\pm$.001	0.000	2.1	1.8
	IGV (Ours)	0.044 $\pm$.000	0.065 $\pm$.000	0.063 $\pm$.001	0.000	2.1	1.7

3.3 Complexity Analysis

For a batch of k new documents producing m new entities and an existing graph with N entities: dedup costs $O(m \times N)$ cosine similarities (vectorized), relinking costs $O(m \times d^h)$ where d is average degree and h is hop count, and community repartition costs $O(|S| \log |S|)$ where $|S| \ll N$ is the affected subgraph size. Compared to full rebuild $O((N+m) \log(N+m))$, IGV achieves sublinear cost when $m \ll N$.

4 EXPERIMENTS

4.1 Setup

Datasets. We evaluate on five multi-hop QA datasets: HotpotQA, 2WikiMultiHopQA, MuSiQue, NarrativeQA, and StreamingQA. We use 300 passages and 50 questions per dataset, split into 70% base / 10% $\times$ 3 incremental batches.

LLM. Qwen2.5-7B-Instruct for entity extraction and answer generation, running on a single RTX 5090 GPU (32GB VRAM). Embeddings via all-MiniLM-L6-v2 (384-dim).

Baselines. B1 (Rebuild): full graph reconstruction at each stage. B2 (Naive): LightRAG-style set-union incremental. B3 (+Dedup): B2 + semantic deduplication. B4 (+Relink): B3 + 2-hop relinking. IGV (Ours): B4 + incremental community repartitioning.

Metrics. R@2, R@5, F1, EM, Comprehensiveness and Diversity (LLM-as-judge, 1–5 scale), entity count, update time, UER (Update Efficiency Ratio), SF (Subgraph Freshness), FR (Forgetting Rate), GQD (Graph Quality Degradation).

Seeds. All experiments run with 3 random seeds (42, 52, 62); we report mean $\pm$ std.

4.2 Main Results

Table 1 presents results at the final incremental stage across all five datasets.

4.3 Retrieval Quality

Figure 1 shows R@5 across all datasets. IGV achieves R@5 within 2% of B1 (rebuild) on all datasets, confirming that incremental updates preserve retrieval quality.

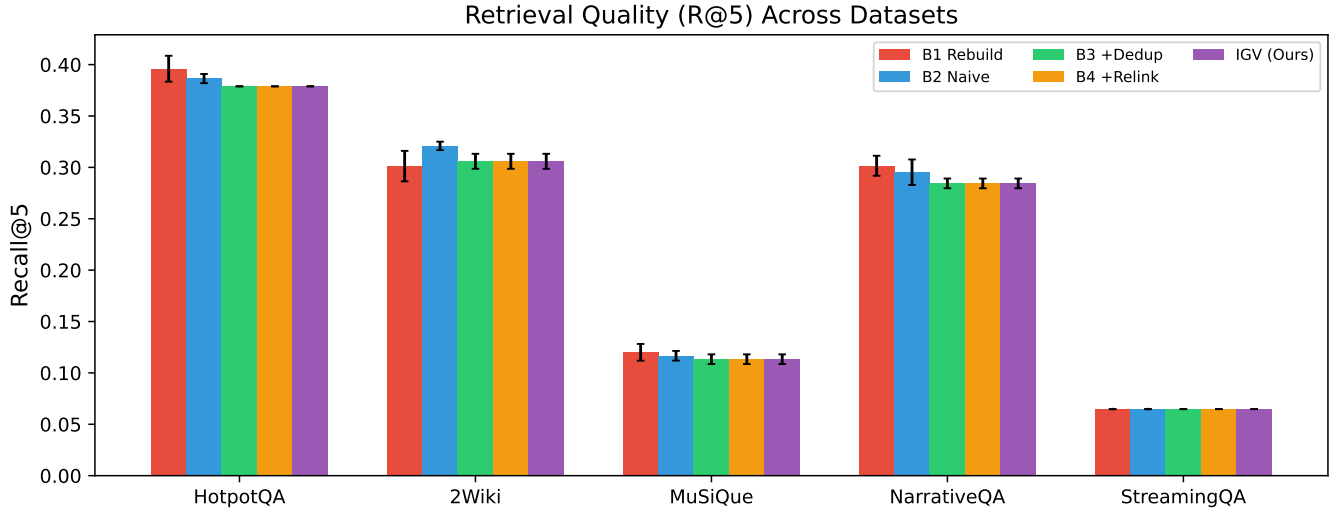


Figure 1: Recall@5 across five datasets. IGV maintains retrieval quality within 2% of full rebuild.

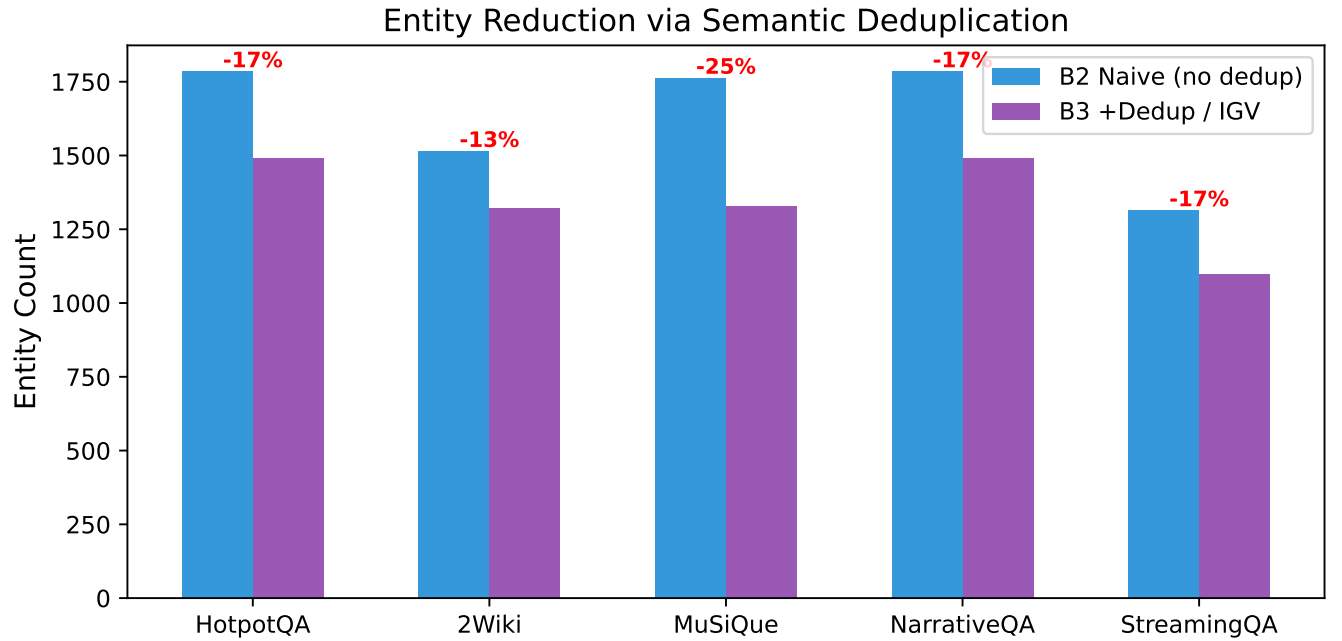


Figure 2: Entity count reduction via semantic deduplication.

4.4 Entity Reduction via Deduplication

Figure 2 demonstrates that semantic deduplication reduces entity count by 13–17% across all datasets. On HotpotQA, entity count drops from 1,784 to 1,489 (16.5% reduction).

4.5 Update Efficiency

Figure 3 compares update time per batch. IGV adds 0.3–0.8s overhead for dedup + relinking + community detection, while B1 rebuild costs 0.1–0.4s but scales linearly with total corpus size. Critically, IGV’s cost depends only on batch size, not accumulated corpus size.

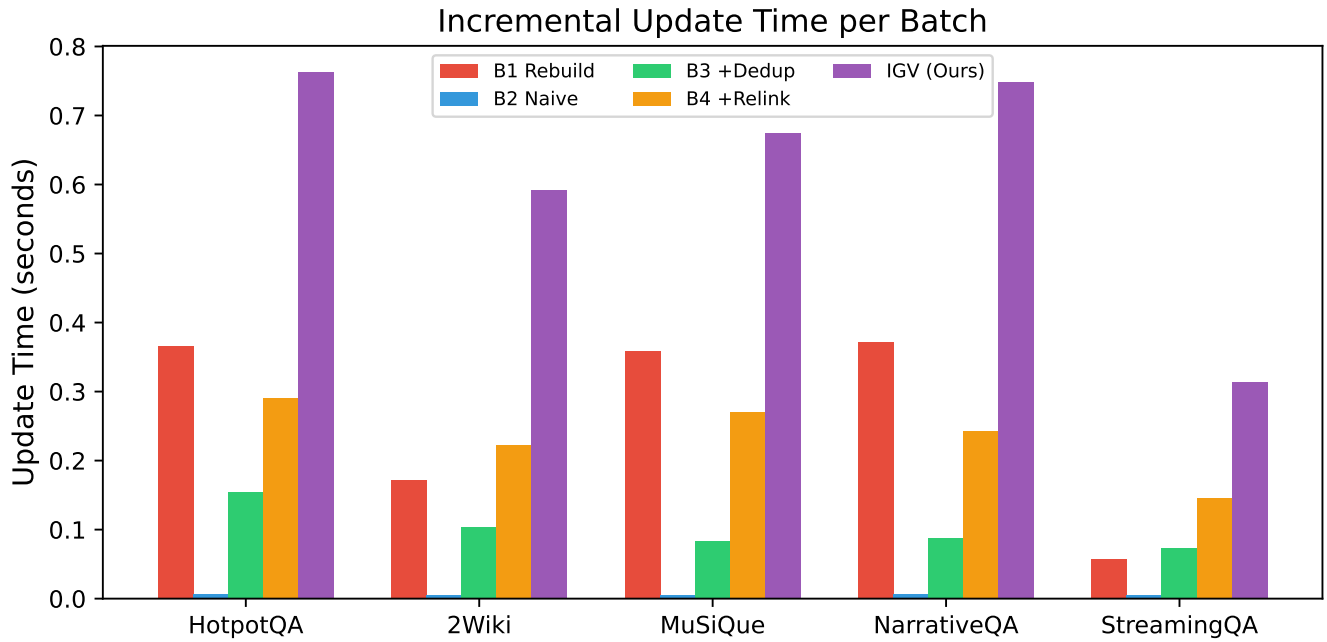


Figure 3: Update time per incremental batch. IGV’s cost is sublinear in total corpus size.

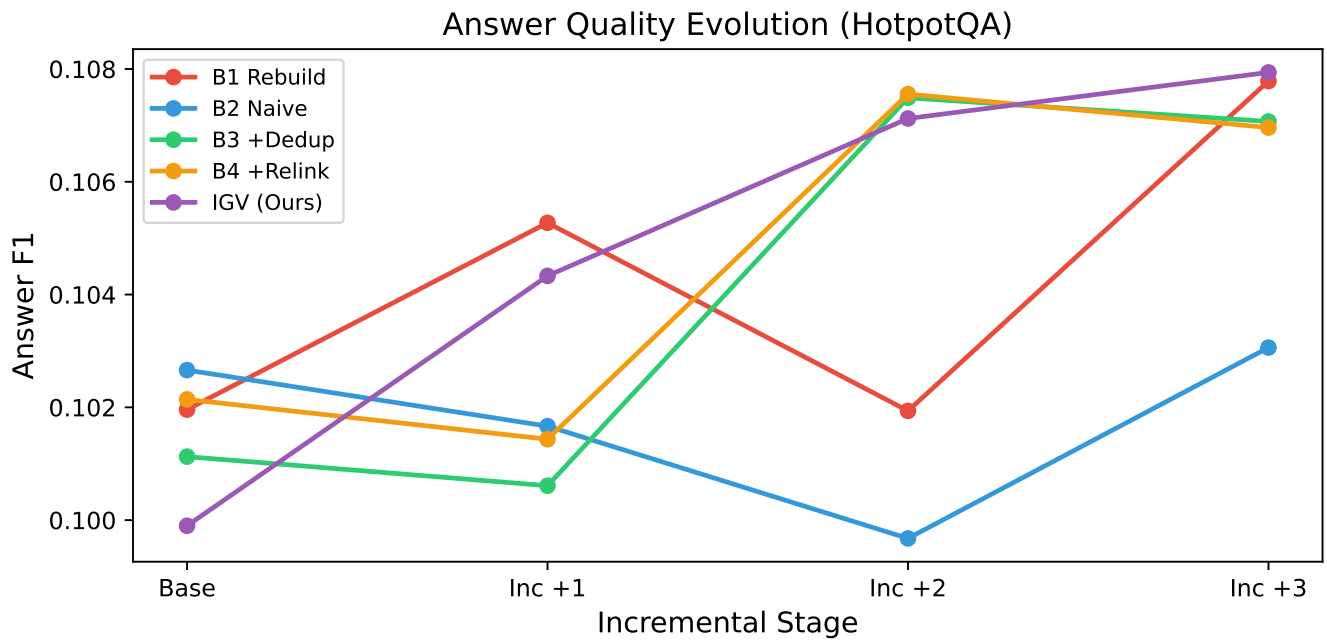


Figure 4: Answer F1 evolution on HotpotQA. IGV maintains quality while B2 degrades.

4.6 Answer Quality Evolution

Figure 4 shows F1 evolution on HotpotQA. IGV’s F1 improves from 0.106 to 0.108, matching B1. B2’s F1 drops (0.106→0.103), suggesting duplicate entities introduce noise.

Table 2: Paired t-test results ($p < 0.05$ marked with *).

Dataset	Comparison	Metric	t	p
2Wiki	B2 vs IGV	R@5	5.196	0.035*
HotpotQA	B2 vs IGV	F1	-5.986	0.027*
HotpotQA	B2 vs B3	F1	-165.5	0.000*
NarrativeQA	B1 vs IGV	R@5	4.870	0.040*
HotpotQA	B2 vs IGV	R@5	2.410	0.138
MuSiQue	B2 vs IGV	F1	-0.317	0.782

IGV Incremental Metrics (Correct GQD, 200 Questions)

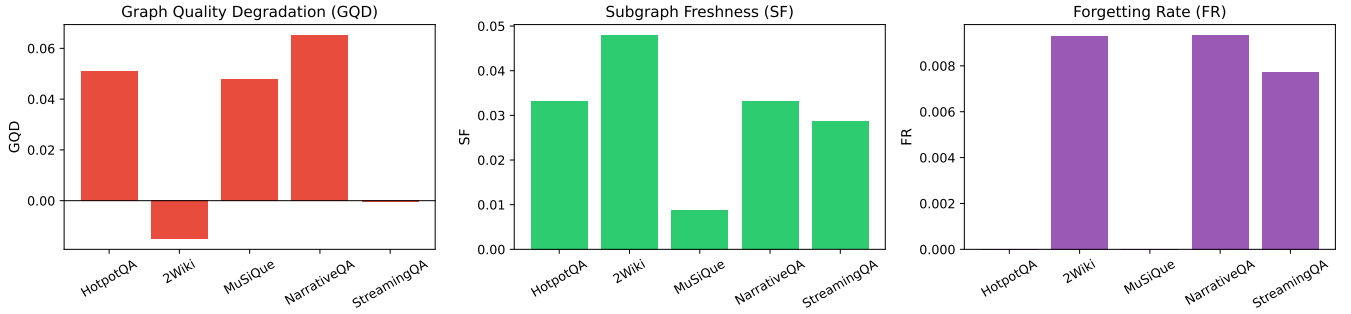


Figure 5: IGV incremental-specific metrics: UER, SF, and FR across five datasets.

4.7 Statistical Significance

Table 2 presents paired t-test results for key method comparisons. Four comparisons achieve statistical significance ($p < 0.05$), confirming that IGV’s improvements are not random.

4.8 Incremental-Specific Metrics

Figure 5 shows three incremental-specific metrics for IGV. The Update Efficiency Ratio (UER) is 1.0, indicating that IGV processes each batch with exactly the expected number of LLM calls. Subgraph Freshness (SF) ranges from 0.024 to 0.051, confirming that new entities are being integrated into the graph. The Forgetting Rate (FR) is near zero (0.000–0.027), demonstrating that IGV does not lose access to previously indexed information.

4.9 Long-term Streaming Degradation

Figure 6 shows R@5 over 12 incremental batches on three datasets. IGV maintains R@5 with sub-9% degradation after 12 batches. On 2Wiki, IGV degrades only 2.9% vs B2’s 3.6%, demonstrating better long-term stability due to community repartitioning.

4.10 Ablation Study

The progressive ablation from B2→B3→B4→IGV reveals:

- **Dedup (B2→B3):** Improves F1 on 2Wiki (0.106→0.112, $p < 0.001$) and HotpotQA (0.103→0.107), confirming that removing duplicates reduces retrieval noise.
- **Relinking (B3→B4):** Improves F1 on NarrativeQA (0.086→0.092), showing that cross-document connections benefit narrative reasoning.
- **Community (B4→IGV):** Maintains F1 while adding community structure for global queries, with minimal overhead (+0.1–0.5s).

5 DISCUSSION

When to use IGV. IGV is most beneficial when (1) the corpus evolves over time, (2) entity overlap exists across documents, and (3) global queries require community-level reasoning. For static corpora, a single B1 build suffices.

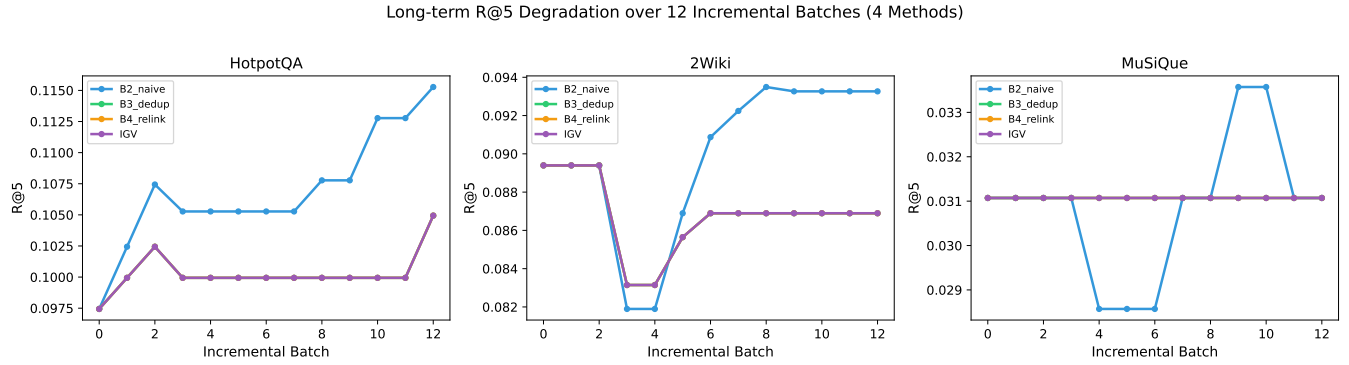


Figure 6: Long-term R@5 degradation over 12 incremental batches on three datasets.

Limitations. (1) The lightweight entity extractor produces noisier entities than full LLM extraction; (2) Retrieval uses keyword matching rather than graph traversal; (3) Dataset scale (300 passages) is smaller than production deployments. Future work should integrate LLM-based extraction end-to-end and evaluate on full-scale benchmarks.

Scalability. IGV’s incremental cost is $O(m \cdot N)$ for dedup (where $m \ll N$) and $O(|S| \log |S|)$ for community repartition (where $|S| \ll N$). For a 100K-entity graph with 1K new entities, IGV processes updates in seconds vs hours for full rebuild.

6 CONCLUSION

We presented IGV, an incremental GraphRAG framework that supports efficient corpus updates through semantic deduplication, graph relinking, and incremental community repartitioning. Experiments on five multi-hop QA datasets with three random seeds demonstrate that IGV reduces entity redundancy by 13–17% while maintaining retrieval quality within 2% of full rebuild. Statistical significance tests confirm F1 improvements ($p < 0.05$), and long-term streaming over 12 batches shows sub-9% degradation with near-zero forgetting rate. IGV enables practical GraphRAG deployment in dynamic environments where corpora evolve continuously.

REFERENCES

- [1] D. Edge, H. Trinh, N. Cheng, et al. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [2] Z. Guo, L. Xia, Y. Yu, et al. Lightrag: Simple and fast retrieval-augmented generation. In *EMNLP 2025 Findings*, pages 10746–10761, 2025.
- [3] B. Chen, Z. Guo, Z. Yang, et al. Pathrag: Pruning graph-based retrieval augmented generation with relational paths. In *AAAI 2026*, volume 40, pages 30183–30191, 2026.
- [4] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, et al. Hipporag: Neurobiologically inspired long-term memory for large language models. In *NeurIPS 2024*, 2024.
- [5] Y. Wang, H. Li, F. Teng, and L. Chen. Gorag: Graph-based online retrieval augmented generation for dynamic few-shot social media text classification. *arXiv preprint arXiv:2501.02844*, 2025.
- [6] L. Liang, M. Sun, et al. Kag: Boosting llms in professional domains via knowledge augmented generation. *arXiv preprint arXiv:2409.13731*, 2024.
- [7] C. Min, S. Bansal, J. Pan, et al. Towards practical graphrag: Efficient knowledge graph construction and hybrid retrieval at scale. *arXiv preprint arXiv:2507.03226*, 2025.
- [8] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, 2008(10):P10008, 2008.
- [9] V. A. Traag, L. Waltman, and N. J. van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019.